

Lume

Banco de dados

Diagrama de relacionamentos e arquitetura de dados

Copyright (©) 2026 Clara A. Portes

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 5

- PostgreSQL é a fonte da verdade 5
- Áreas do modelo 5
- Banco suportado 5
- O que fica no disco 5
- O que fica no Redis 6

Diagrama de relacionamentos 7

- Acesso 8
- Biblioteca 9
- Leitura 10
- Processamento 11
- Tabelas do framework 11
- Cardinalidades 11

Regras e convenções 13

- Convenções do modelo 13
- Integridade e concorrência 13
- Tipos usados 14

Tabelas 15

- accounts_user 15
- accounts_apitoken 16
- library_document 16
- library_documentcontent 19
- library_tag 19
- library_documenttag 20
- library_collection 20
- library_collectionitem 21

library_readingstate 22
library_annotation 23
processing_job 24

Máquinas de estado 25

Documento 25
Job 26
Estante 27

Busca em texto completo 28

Como o vetor é calculado 28
Como a busca consulta 28
Trechos encontrados 28

Um documento passo a passo 30

1. Envio 30
2. Leitura do arquivo (processador Java) 30
3. Descrição por IA 30
Chaves do Redis 30

Consultas úteis 32

Documentos por estado 32
Falhas recentes 32
Tempo médio por etapa nas últimas 24 horas 32
Maiores arquivos 32
Leitura por pessoa 32
Etiquetas mais usadas 33
Documentos parados há mais de 30 minutos 33

Como preparar o banco 34

Com Docker (recomendado) 34
PostgreSQL próprio 34

Banco gerenciado 34
Redis 35

Backup, retenção e problemas comuns 36

Backup 36
Restauração 36
Retenção e tamanho 36
Problemas comuns 36

Evolução do esquema 38

CAPÍTULO 1

Visão geral

O Lume guarda tudo o que importa no PostgreSQL: contas, documentos, metadados, texto extraído, descrições, etiquetas, coleções, progresso de leitura, anotações e o histórico do processamento. O Redis só transporta trabalho e avisos; os arquivos ficam num volume de disco.

PostgreSQL é a fonte da verdade

Se o Redis for apagado, nada se perde: o worker recria os grupos das filas e uma varredura periódica encontra documentos parados no meio do caminho e os coloca na fila de novo. Se o processador Java ou um worker cair durante uma etapa, o estado gravado no banco diz exatamente o que falta fazer. Por isso todo backup começa pelo PostgreSQL e pelo volume de arquivos (capítulo 10).

Áreas do modelo

Área	Tabelas	O que guarda
Acesso	`accounts_user`, `accounts_apitoken`	Contas, papéis, preferências e tokens pessoais da API.
Biblioteca	`library_document`, `library_documentcontent`, `library_tag`, `library_documenttag`, `library_collection`, `library_collectionitem`	Os documentos com metadados, capa, sumário, texto, descrição por IA, busca, etiquetas e coleções.
Leitura	`library_readingstate`, `library_annotation`	Estante, página, progresso, favoritos e anotações de cada pessoa.
Processamento	`processing_job`	Cada tentativa de cada etapa do pipeline, com duração e erro.

Banco suportado

PostgreSQL 16 (é o que o `docker-compose.yml` usa; versões 14 ou mais novas também funcionam). A primeira migração do app `library` cria o que a busca precisa:

- **pg_trgm**: similaridade por trigramas, para achar títulos mesmo com erro de digitação.
- **unaccent**: remove acentos ao indexar e ao buscar (*decisões* encontra *decisoes*).
- **pt_unaccent**: configuração de busca textual copiada de `portuguese`, com `unaccent` antes do radical português nas palavras.

O que fica no disco

Os arquivos ficam no volume compartilhado `/data` (volume `documents` no Docker), que o web, o worker e o processador Java montam. Cada documento tem uma pasta com o mesmo nome do seu `id`:

Caminho	Quem grava	Conteúdo
`documents/<id>/original.pdf` ou `.docx`	web	O arquivo enviado, sem alteração.
`documents/<id>/cover.jpg`	processador	Capa (página 1 do PDF ou capa tipográfica do DOCX), 480 px de largura.
`documents/<id>/reader.html`	processador	HTML de leitura gerado do DOCX (antes da higienização).
`documents/<id>/reader.safe.html`	worker	O mesmo HTML depois da lista de permissões; é o que o navegador recebe.
`documents/<id>/media/`	processador	Imagens extraídas do DOCX.
`tmp/*.part`	web	Envio em andamento; apagado ao terminar ou falhar.

Excluir um documento apaga a pasta inteira e as linhas ligadas a ele. O banco guarda só o caminho implícito (pelo `id`), nunca o binário.

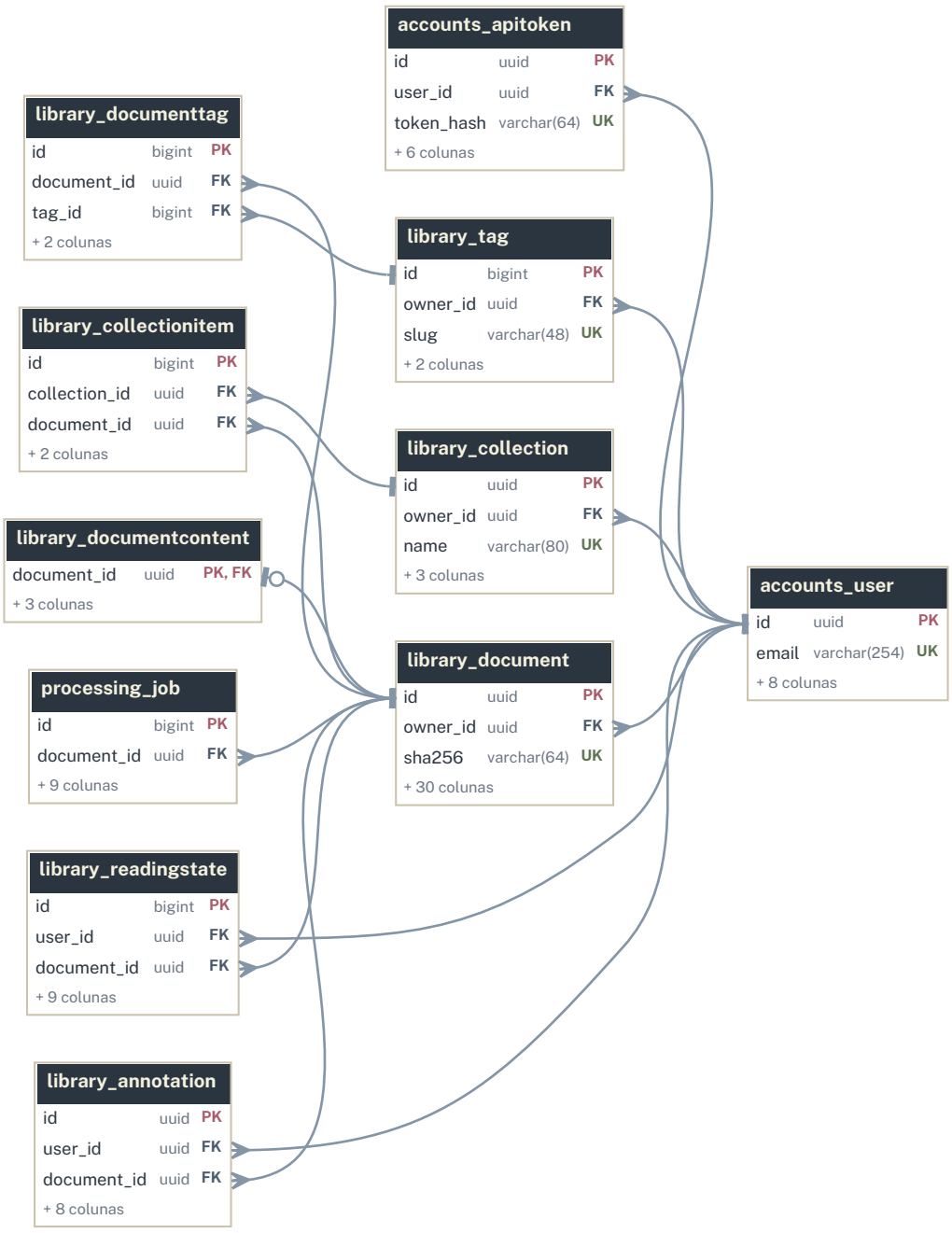
O que fica no Redis

Filas de trabalho (Redis Streams), retentativas agendadas, eventos em tempo real (pub/sub), batimento dos workers, contador de tentativas de login, cache e cópia das sessões. Tudo isso pode ser perdido sem prejuízo permanente; a lista completa de chaves está no capítulo 7.

CAPÍTULO 2

Diagrama de relacionamentos

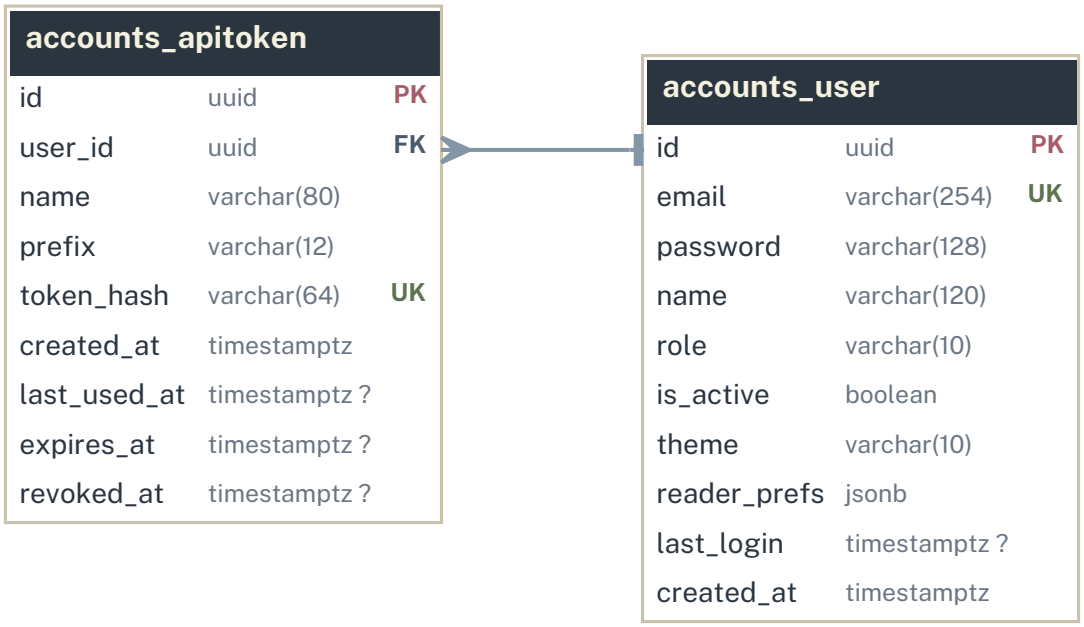
As ligações são chaves estrangeiras de verdade. O pé de galinha fica do lado “muitos” e o traço do lado “um”; o círculo marca o lado opcional da relação 1:0..1 entre o documento e o seu texto. Todas as chaves estrangeiras são obrigatórias (nenhuma aceita nulo). Nas tabelas com muitas colunas, o diagrama geral mostra só as chaves; as demais aparecem como “+ N colunas”.



Todas as tabelas do Lume, com chaves primárias (PK), estrangeiras (FK) e únicas (UK)

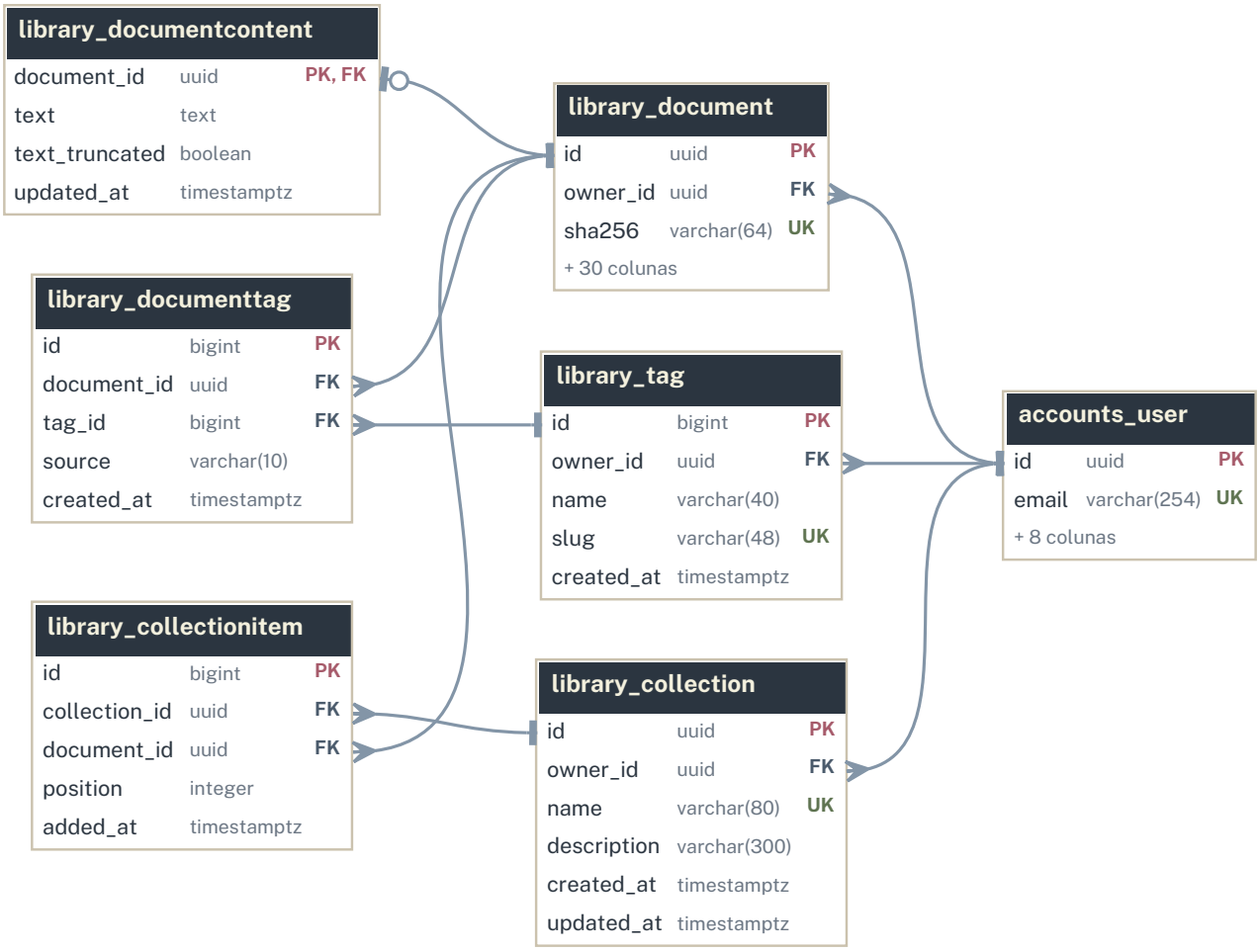
Acesso

Uma conta tem vários tokens. O token guarda só o hash; a autenticação calcula o SHA-256 do valor recebido e procura em `token_hash`.



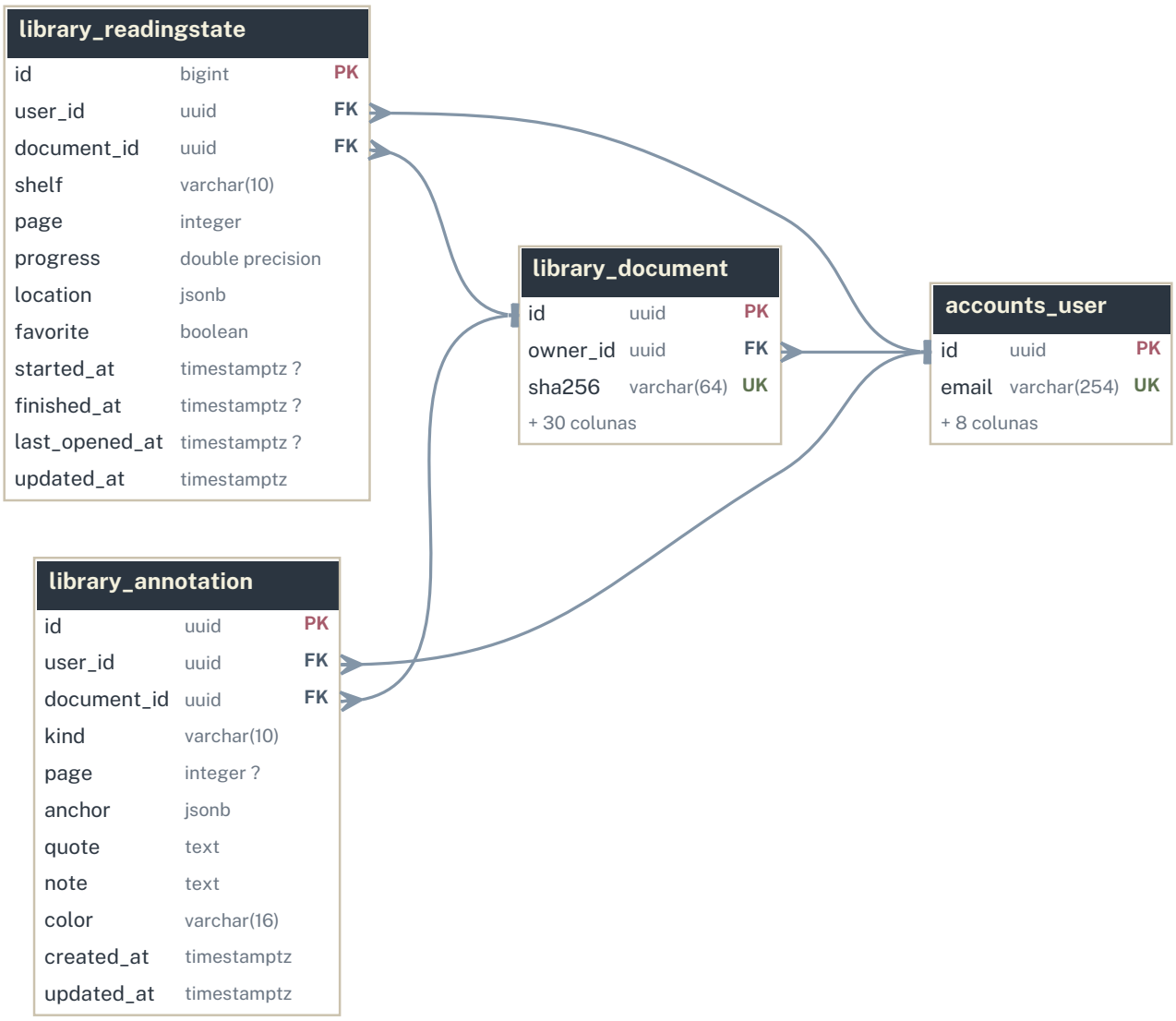
Biblioteca

O documento pertence a uma conta. O texto completo fica numa tabela à parte, ligada 1:1. Etiquetas e coleções são ligações N:N com tabelas intermediárias, que guardam dados da própria ligação (quem sugeriu a etiqueta; a posição na coleção).



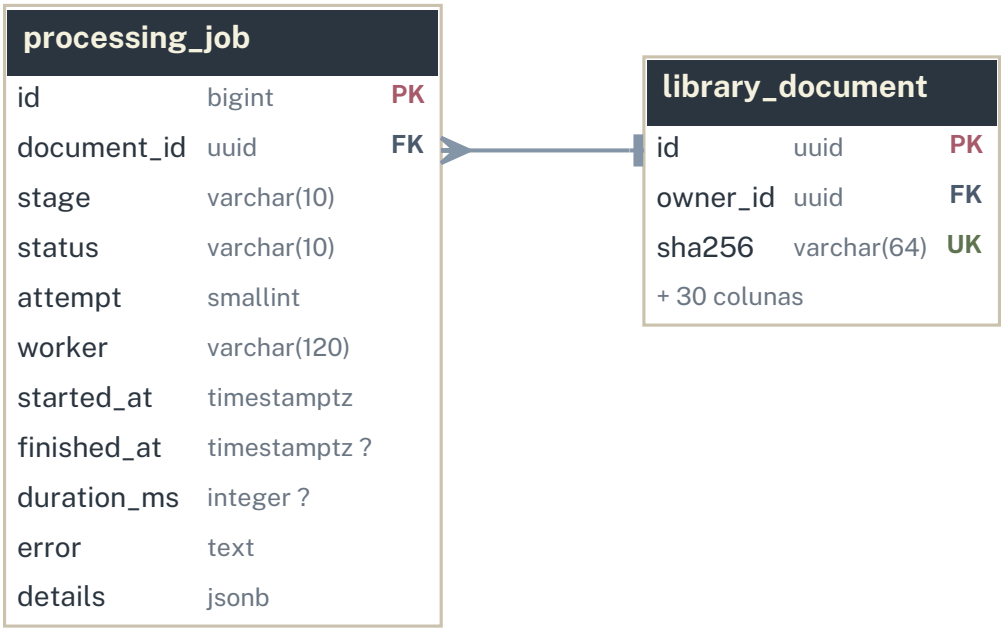
Leitura

Estado de leitura e anotações são da pessoa, não do documento: num documento compartilhado, cada leitor tem a sua estante, a sua página e as suas anotações.



Processamento

Cada etapa de cada tentativa vira uma linha em `processing_job`.



Tabelas do framework

O Django cria e mantém estas tabelas; o Lume não grava nelas diretamente:

Tabela	Uso
`django_session`	Sessões do navegador (o motor `cached_db` grava aqui e guarda uma cópia no Redis).
`django_migrations`	Quais migrações já foram aplicadas.
`django_content_type`	Registro dos modelos, usado pelo sistema de permissões do Django.
`auth_permission`, `auth_group`, `auth_group_permissions`	Criadas pelo app de autenticação do Django. O Lume usa o campo `role` da conta e não usa grupos; as tabelas ficam vazias (exceto as permissões padrão).

Cardinalidades

Tabela “um”	Relação	Tabela “muitos”	Pela coluna
`accounts_user`	1 : N	`accounts_apitoken`	`accounts_apitoken.user_id`
`accounts_user`	1 : N	`library_document`	`library_document.owner_id`
`accounts_user`	1 : N	`library_tag`	`library_tag.owner_id`
`accounts_user`	1 : N	`library_collection`	`library_collection.owner_id`
`accounts_user`	1 : N	`library_readingstate`	`library_readingstate.user_id`
`accounts_user`	1 : N	`library_annotation`	`library_annotation.user_id`
`library_document`	1 : 0..1	`library_documentcontent`	`library_documentcontent.document_id`

Tabela “um”	Relação	Tabela “muitos”	Pela coluna
`library_document`	1 : N	`library_documenttag`	`library_documenttag.document_id`
`library_tag`	1 : N	`library_documenttag`	`library_documenttag.tag_id`
`library_collection`	1 : N	`library_collectionitem`	`library_collectionitem.collection_id`
`library_document`	1 : N	`library_collectionitem`	`library_collectionitem.document_id`
`library_document`	1 : N	`library_readingstate`	`library_readingstate.document_id`
`library_document`	1 : N	`library_annotation`	`library_annotation.document_id`
`library_document`	1 : N	`processing_job`	`processing_job.document_id`

Pelas restrições únicas: cada conta tem no máximo um estado de leitura por documento; cada documento aparece uma vez em cada coleção e recebe cada etiqueta uma vez.

CAPÍTULO 3

Regras e convenções

Convenções do modelo

- **Nomes:** `<app>_<modelo>`, em minúsculas e sem acento: `accounts_user`, `library_document`. Chaves estrangeiras terminam em `_id`.
- **Chaves:** as entidades que aparecem em endereços e na API (contas, tokens, documentos, coleções e anotações) usam **UUID** gerado pela aplicação; ninguém consegue adivinhar o próximo. Tabelas de ligação e de histórico usam `bigint` com `GENERATED BY DEFAULT AS IDENTITY`.
- **Datas:** sempre `timestamptz`, gravadas em UTC. A interface converte para o fuso de `APP_TIMEZONE` (padrão `America/Sao_Paulo`).
- **Textos opcionais** são `''` (vazio) e não nulo, como manda a convenção do Django. Nulo fica para números e datas que podem não existir (páginas de um arquivo ainda não lido, fim de uma etapa em andamento).
- **Valores padrão** (a coluna “Padrão” do capítulo 4) são aplicados pela aplicação ao criar o registro; o banco não tem `DEFAULT`. Ao inserir linhas à mão, informe todas as colunas não nulas.
- **Listas e objetos** ficam em `jsonb` com padrão `[]` ou `{}`, nunca nulo.
- **Escolhas** (estado, papel, estante, cor) são `varchar` com valores fixos validados pela aplicação.

Integridade e concorrência

Regra	Como é garantida
Um e-mail por conta, sem diferenciar maiúsculas	Índice único em `lower(email)` e e-mail gravado em minúsculas.
Mesmo arquivo não entra duas vezes na biblioteca de uma pessoa	`UNIQUE (owner_id, sha256)`. O envio repetido devolve o documento existente com `duplicate: true`.
Uma etiqueta por nome e dono	`UNIQUE (owner_id, slug)`.
Nomes de coleção sem repetição	`UNIQUE (owner_id, lower(name))`.
Um estado de leitura por pessoa e documento	`UNIQUE (user_id, document_id)`.
Progresso entre 0 e 1	`CHECK (progress >= 0 AND progress <= 1)`.
Contadores nunca negativos	`CHECK (… >= 0)` nas colunas inteiras sem sinal (páginas, tentativas, posição, duração).
Duas cópias do worker não processam o mesmo documento	A etapa começa com `SELECT … FOR UPDATE SKIP LOCKED` e só segue se o estado for o esperado; mensagem repetida vira “skipped”.
A fila só recebe o documento depois que a linha existe	O `XADD` é feito em `transaction.on_commit`, depois do `COMMIT`.

EXCLUSÕES EM CASCATA

As chaves estrangeiras são `DEFERRABLE INITIALLY DEFERRED` e **não** têm `ON DELETE CASCADE` no banco: quem apaga em cascata é o Django (`on_delete=CASCADE`). Excluir uma conta pela aplicação apaga os documentos, o texto, as etiquetas, as coleções, as leituras, as anotações, os jobs e as pastas no disco. Um `DELETE` feito direto no `psql` falha se houver linhas filhas; apague as filhas antes ou use a API/o painel.

Tipos usados

Tipo	Onde
`uuid`	Chaves de contas, tokens, documentos, coleções e anotações.
`bigint`	Chaves de etiquetas, ligações, leituras e jobs; `size_bytes`.
`integer` / `smallint`	Páginas, palavras, medidas da capa, posição, duração; tentativas.
`double precision`	`progress` do estado de leitura.
`varchar(n)`	Textos curtos com limite: títulos, nomes, escolhas, hashes, cores.
`text`	Textos longos: descrição, texto extraído, notas, erros.
`boolean`	Marcadores (ativo, favorito, capa gerada…).
`jsonb`	Sumário, metadados, âncoras, posição, preferências e detalhes das etapas.
`tsvector`	`search_vector` da busca em texto completo.
`timestamptz`	Todas as datas.

CAPÍTULO 4

Tabelas

Uma seção por tabela, na ordem das áreas. “Nulo” indica se a coluna aceita `NULL`; “Padrão” é o valor que a aplicação grava ao criar o registro.

accounts_user

ACESSO

Contas de quem usa a instalação. O login é pelo e-mail; o papel define quem administra.

É referenciada por: `accounts_apitoken.user_id`, `library_document.owner_id`, `library_tag.owner_id`, `library_collection.owner_id`, `library_readingstate.user_id`, `library_annotation.user_id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	`uuid`	não	`uuid4()`	Identificador da conta.
email UK	`varchar(254)`	não	–	E-mail de acesso, gravado em minúsculas. Único sem diferenciar maiúsculas (índice sobre lower(email)).
password	`varchar(128)`	não	–	Hash da senha no formato do Django (PBKDF2-SHA256 com sal e iterações). A senha em si nunca é gravada.
name	`varchar(120)`	não	–	Nome exibido no menu, nos documentos compartilhados e no painel.
role	`varchar(10)`	não	`'member'`	admin ou member. Administradores veem o painel Sistema e gerenciam contas.
is_active	`boolean`	não	`true`	Conta desativada não entra, não usa tokens e some do WebSocket.
theme	`varchar(10)`	não	`'system'`	system, light ou dark.
reader_prefs	`jsonb`	não	`{}`	Preferências do leitor (cor da página e tamanho da letra).
last_login	`timestamptz`	sim	–	Último login pela tela de entrada.
created_at	`timestamptz`	não	`now()`	Criação da conta.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
`accounts_user_pkey`	`PRIMARY KEY (id)`	Chave primária.
`accounts_user_email_unique`	`UNIQUE (lower(email))`	Impede duas contas com o mesmo e-mail escrito de formas diferentes.

EXEMPLOS DE JSON

Campo	Exemplo	Quando
reader_prefs	`{"theme": "paper", "fontSize": 19}`	Gravado pelo leitor quando a pessoa troca a cor da página ou o tamanho da letra.

accounts_apitoken

ACESSO

Tokens pessoais para scripts e integrações (cabeçalho Authorization: Bearer). Só o hash é gravado; o valor aparece uma única vez, na criação.

Aponta para: `user_id` → `accounts_user.id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	uuid	não	uuid4()	Identificador do token.
user_id FK	uuid	não	–	Dono do token → accounts_user.id.
name	varchar(80)	não	–	Para que o token serve (ex.: importação noturna).
prefix	varchar(12)	não	–	8 caracteres hexadecimais visíveis no token (dl_<prefixo>_<segredo>) e na tela, para reconhecê-lo.
token_hash UK	varchar(64)	não	–	SHA-256 do token completo. A autenticação procura por este campo.
created_at	timestamptz	não	now()	Criação.
last_used_at	timestamptz	sim	–	Último uso, atualizado no máximo uma vez por minuto.
expires_at	timestamptz	sim	–	Validade opcional.
revoked_at	timestamptz	sim	–	Preenchido ao revogar. O registro fica para histórico.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
accounts_apitoken_pkey	PRIMARY KEY (id)	Chave primária.
accounts_apitoken_token_hash_key	UNIQUE (token_hash)	Busca do token na autenticação.
accounts_apitoken_prefix_…	btree (prefix)	Consulta pelo prefixo exibido.
accounts_apitoken_user_id_…	btree (user_id)	Tokens de uma conta.

library_document

BIBLIOTECA

Um arquivo enviado (PDF ou DOCX) com tudo o que o pipeline descobriu sobre ele: metadados, capa, sumário, descrição por IA, estado do processamento e vetor de busca. O texto completo fica em library_documentcontent.

Aponta para: `owner_id` → `accounts_user.id`. É referenciada por: `library_documentcontent.document_id`, `library_documenttag.document_id`, `library_collectionitem.document_id`, `library_readingstate.document_id`, `library_annotation.document_id`, `processing_job.document_id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	uuid	não	uuid4()	Identificador. Também é o nome da pasta no disco: /data/documents/<id>/.

Coluna	Tipo	Nulo	Padrão	Descrição
owner_id FK	`uuid`	não	–	Quem enviou → accounts_user.id.
title	`varchar(300)`	não	–	Começa com o nome do arquivo e é trocado pelo título dos metadados quando ele é bom (e o usuário não editou).
author	`varchar(200)`	não	`''`	Autor dos metadados ou digitado pelo usuário.
description	`text`	não	`''`	Descrição escrita pelo usuário (opcional). Entra na busca.
metadata_locked	`boolean`	não	`false`	true quando título ou autor foram editados à mão: os metadados do arquivo deixam de sobrescrever.
summary	`text`	não	`''`	Descrição gerada pela IA (2 a 3 frases).
summary_source	`varchar(20)`	não	`''`	Quem escreveu: local, anthropic ou openai.
summary_model	`varchar(100)`	não	`''`	Modelo usado (vazio no resumo local).
summary_at	`timestamptz`	sim	–	Quando a descrição foi gerada.
kind	`varchar(8)`	não	–	pdf ou docx, detectado pelo conteúdo do arquivo (não pela extensão).
original_name	`varchar(255)`	não	–	Nome do arquivo enviado; usado no download.
size_bytes	`bigint`	não	–	Tamanho em bytes.
sha256 UK	`varchar(64)`	não	–	Impressão digital do arquivo. Com owner_id, impede enviar o mesmo arquivo duas vezes.
page_count	`integer`	sim	–	Páginas (PDF) ou estimativa (DOCX: app.xml ou palavras ÷ 350).
word_count	`integer`	sim	–	Palavras do texto extraído.
language	`varchar(16)`	não	`''`	Idioma dos metadados (pt-BR) ou detectado no texto (pt, en, es).
cover_width	`integer`	sim	–	Largura da capa gerada, em pixels.
cover_height	`integer`	sim	–	Altura da capa gerada, em pixels.
cover_color	`varchar(7)`	não	`''`	Cor média da capa (#rrggbb). Fundo do cartão enquanto a imagem carrega.
cover_generated	`boolean`	não	`false`	true para a capa tipográfica dos DOCX.
cover_version	`integer`	não	`0`	Sobe a cada capa nova; 0 = sem capa. Vai no endereço da imagem (?v=) para renovar o cache.
has_reader_html	`boolean`	não	`false`	true quando existe reader.safe.html (DOCX).
outline	`jsonb`	não	`[]`	Sumário do documento (até 500 itens).

Coluna	Tipo	Nulo	Padrão	Descrição
metadata	`jsonb`	não	`{}`	Metadados técnicos do arquivo devolvidos pelo processador.
status	`varchar(12)`	não	`'queued'`	queued, extracting, describing, ready ou failed.
status_detail	`text`	não	`''`	Motivo da falha ou aviso de nova tentativa ("Nova tentativa em 15 s").
attempts	`smallint`	não	`0`	Tentativas da etapa atual. Volta a 0 quando a etapa termina.
visibility	`varchar(10)`	não	`'private'`	private (só o dono) ou shared (todas as contas da instalação).
search_vector	`tsvector`	sim	–	Vetor da busca em texto completo, calculado pelo pipeline.
created_at	`timestamptz`	não	`now()`	Envio.
updated_at	`timestamptz`	não	`now()`	Última alteração (atualizado a cada gravação pelo Django).
processed_at	`timestamptz`	sim	–	Primeira vez em que o documento ficou pronto.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
`library_document_pkey`	`PRIMARY KEY (id)`	Chave primária.
`library_document_owner_sha256_unique`	`UNIQUE (owner_id, sha256)`	Deduplicação por dono.
`library_doc_owner_created`	`btree (owner_id, created_at DESC)`	Biblioteca de uma pessoa, mais recentes primeiro.
`library_doc_shared`	`btree (visibility, created_at DESC) WHERE visibility = 'shared'`	Índice parcial: só os compartilhados.
`library_doc_search`	`GIN (search_vector)`	Busca em texto completo.
`library_doc_title_trgm`	`GIN (title gin_trgm_ops)`	Similaridade por trigramas no título (erros de digitação).
`library_document_status_…`	`btree (status)`	Painel do Sistema e varredura de documentos parados.
`library_document_owner_id_…`	`btree (owner_id)`	Chave estrangeira.

Verificações: attempts, page_count, word_count, cover_width, cover_height e cover_version ≥ 0.

OUTLINE (PDF)

Marcadores do PDF com a página de cada um. level vai de 1 a 4.

```
[
  {"title": "Sobre este material", "page": 2, "level": 1, "anchor": null},
  {"title": "Tipos mais usados", "page": 5, "level": 2, "anchor": null}
]
```

OUTLINE (DOCX)

Títulos do Word; anchor é o id do título no HTML de leitura.


```
[
  {"title": "1. Objetivo", "page": null, "level": 1, "anchor": "s-2"},
  {"title": "Jornada e horários", "page": null, "level": 2, "anchor": "s-6"}
]
```


METADATA

Só as chaves que vieram preenchidas do processador Java.


```
{"subject": "Apresentação de resultados trimestrais",
 "keywords": "resultados, trimestre, vendas",
 "producer": "Skia/PDF", "creator": "Documento de exemplo",
 "created_at": "2026-09-15T13:00:00Z", "modified_at": "2026-10-02T15:00:00Z",
 "page_width": 960.0, "page_height": 540.0, "encrypted": false,
 "char_count": 2381, "text_truncated": false, "elapsed_ms": 214, "warnings": []}
```


---

library_documentcontent

BIBLIOTECA

Texto completo extraído, separado da tabela principal para manter as listagens leves (uma linha por documento).

Aponta para: `document_id` → `library_document.id`.

Coluna	Tipo	Nulo	Padrão	Descrição
document_id PK FK	uuid	não	—	Documento → library_document.id (relação 1:1).
text	text	não	''	Texto extraído. No PDF, as páginas são separadas pelo caractere \f (form feed). Limite: 2 milhões de caracteres.
text_truncated	boolean	não	false	true quando o texto passou do limite e foi cortado.
updated_at	timestamptz	não	now()	Última extração.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
library_documentcontent_pkey	PRIMARY KEY (document_id)	Chave primária e estrangeira ao mesmo tempo.

---

library_tag

BIBLIOTECA

Etiquetas de cada pessoa. A mesma etiqueta pode ser usada em vários documentos do dono.

Aponta para: `owner_id` → `accounts_user.id`. É referenciada por: `library_documenttag.tag_id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	bigint	não	identity	Identificador.

Coluna	Tipo	Nulo	Padrão	Descrição
owner_id FK	uuid	não	–	Dono → accounts_user.id.
name	varchar(40)	não	–	Nome em minúsculas (ex.: banco de dados).
slug UK	varchar(48)	não	–	Versão para endereços (banco-de-dados). Único por dono.
created_at	timestamptz	não	now()	Criação.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
library_tag_pkey	PRIMARY KEY (id)	Chave primária.
library_tag_owner_slug_unique	UNIQUE (owner_id, slug)	Uma etiqueta por nome e dono.
library_tag_slug_…	btree (slug)	Filtro ?tag= da biblioteca.

library_documenttag

BIBLIOTECA

Liga documentos e etiquetas (N:N) e guarda quem sugeriu a ligação: a IA ou a pessoa.

Aponta para: `document_id` → `library_document.id`, `tag_id` → `library_tag.id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	bigint	não	identity	Identificador.
document_id FK	uuid	não	–	→ library_document.id.
tag_id FK	bigint	não	–	→ library_tag.id.
source	varchar(10)	não	'usuario'	ia (sugerida na descrição) ou usuario. Gerar a descrição de novo troca só as da IA.
created_at	timestamptz	não	now()	Criação.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
library_documenttag_unique	UNIQUE (document_id, tag_id)	A mesma etiqueta não entra duas vezes no documento.
…_document_id / …_tag_id	btree	Chaves estrangeiras.

library_collection

BIBLIOTECA

Coleções criadas pelo usuário para agrupar documentos (um documento pode estar em várias).

Aponta para: `owner_id` → `accounts_user.id`. É referenciada por: `library_collectionitem.collection_id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	`uuid`	não	`uuid4()`	Identificador.
owner_id FK	`uuid`	não	—	Dono → accounts_user.id.
name UK	`varchar(80)`	não	—	Nome. Único por dono, sem diferenciar maiúsculas.
description	`varchar(300)`	não	`''`	Descrição opcional.
created_at	`timestamptz`	não	`now()`	Criação.
updated_at	`timestamptz`	não	`now()`	Última alteração (inclui documentos adicionados).

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
`library_collection_owner_name_unique`	`UNIQUE (owner_id, lower(name))`	Sem duas coleções com o mesmo nome.
`library_collection_owner_id_…`	`btree (owner_id)`	Chave estrangeira.

`library_collectionitem`

BIBLIOTECA

Documentos de cada coleção, com a ordem escolhida ao arrastar os cartões.

Aponta para: `collection_id` → `library_collection.id`, `document_id` → `library_document.id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	`bigint`	não	`identity`	Identificador.
collection_id FK	`uuid`	não	—	→ library_collection.id.
document_id FK	`uuid`	não	—	→ library_document.id (pode ser um documento compartilhado de outra pessoa).
position	`integer`	não	`0`	Ordem dentro da coleção (≥ 0).
added_at	`timestamptz`	não	`now()`	Quando entrou na coleção.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
`library_collectionitem_unique`	`UNIQUE (collection_id, document_id)`	O documento entra uma vez em cada coleção.

library_readingstate

LEITURA

O estado de leitura de cada pessoa em cada documento: estante, página, progresso, favorito e última abertura. É pessoal: num documento compartilhado, cada leitor tem a sua linha.

Aponta para: `user_id` → `accounts_user.id`, `document_id` → `library_document.id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	bigint	não	identity	Identificador.
user_id FK	uuid	não	—	Leitor → accounts_user.id.
document_id FK	uuid	não	—	→ library_document.id.
shelf	varchar(10)	não	''	Estante: vazio, para_ler, lendo ou lido.
page	integer	não	1	Página atual (PDF). Limitada ao número de páginas.
progress	double precision	não	0	Fração lida, de 0 a 1 (restrição de verificação).
location	jsonb	não	{}	Posição no DOCX (bloco do texto).
favorite	boolean	não	false	Marcado com estrela.
started_at	timestamptz	sim	—	Quando foi para "lendo" pela primeira vez.
finished_at	timestamptz	sim	—	Quando foi marcado como lido.
last_opened_at	timestamptz	sim	—	Última abertura no leitor (alimenta o "Continuar lendo").
updated_at	timestamptz	não	now()	Última alteração.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
library_readingstate_unique	UNIQUE (user_id, document_id)	Uma linha por leitor e documento.
library_reading_recent	btree (user_id, last_opened_at DESC)	Continuar lendo e ordenação "lidos recentemente".

Verificações: progress entre 0 e 1 (library_readingstate_progress); page ≥ 0.

EXEMPLOS DE JSON

Campo	Exemplo	Quando
location	`{"block": 12}`	Só no DOCX: índice do bloco (atributo data-b do HTML) que estava no topo da tela.

library_annotation

LEITURA

Destaques, notas e marcadores feitos no leitor. Também são pessoais.

Aponta para: `user_id` → `accounts_user.id`, `document_id` → `library_document.id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	uuid	não	uuid4()	Identificador.
user_id FK	uuid	não	–	Autor → accounts_user.id.
document_id FK	uuid	não	–	→ library_document.id.
kind	varchar(10)	não	–	highlight (destaque), note (nota) ou bookmark (marcador).
page	integer	sim	–	Página do PDF. Nulo no DOCX.
anchor	jsonb	não	{}	Onde a anotação fica: retângulos na página (PDF) ou bloco do texto (DOCX).
quote	text	não	''	Trecho selecionado (até 4.000 caracteres).
note	text	não	''	Texto da nota (até 8.000 caracteres; obrigatório quando kind = note).
color	varchar(16)	não	'salvia'	salvia, rose, ardosia ou camurca.
created_at	timestamptz	não	now()	Criação.
updated_at	timestamptz	não	now()	Última alteração.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
library_annotation_user_doc	btree (user_id, document_id)	Anotações de uma pessoa num documento (abertura do leitor).

Verificações: page ≥ 0. A API limita 2.000 anotações por pessoa em cada documento.

EXEMPLOS DE JSON

Campo	Exemplo	Quando
anchor (PDF)	`{"rects": [[0.099, 0.119, 0.802, 0.0175], [0.099, 0.1385, 0.18, 0.0175]]}`	Retângulos do trecho, em frações da página (x, y, largura, altura), para funcionar em qualquer zoom.
anchor (DOCX)	`{"block": 7}`	Bloco do texto onde o trecho começa; o destaque é refeito procurando quote dentro dele.
anchor (marcador)	`{}`	Marcador de página não precisa de âncora (a página está em page).

processing_job

PROCESSAMENTO

Histórico de cada tentativa de cada etapa do pipeline. Alimenta o painel Sistema e ajuda a investigar falhas.

Aponta para: `document_id` → `library_document.id`.

Coluna	Tipo	Nulo	Padrão	Descrição
id PK	bigint	não	identity	Identificador.
document_id FK	uuid	não	–	→ library_document.id.
stage	varchar(10)	não	–	extract (leitura pelo processador Java) ou describe (descrição por IA).
status	varchar(10)	não	'running'	running, succeeded, retrying ou failed.
attempt	smallint	não	1	Número da tentativa desta etapa.
worker	varchar(120)	não	''	Consumidor que executou (host-pid-thread).
started_at	timestamptz	não	now()	Início.
finished_at	timestamptz	sim	–	Fim.
duration_ms	integer	sim	–	Duração em milissegundos.
error	text	não	''	Erro (até 4.000 caracteres) ou aviso, como a queda para o resumo local.
details	jsonb	não	{}	Números da etapa: páginas, palavras, tempo no processador, provedor e modelo de IA, espera da nova tentativa.

ÍNDICES E RESTRIÇÕES

Índice ou restrição	Definição	Para quê
processing_job_document	btree (document_id, started_at DESC)	Histórico de um documento.
processing_job_status	btree (status, started_at DESC)	Painel: últimas etapas e falhas.

Verificações: attempt ≥ 0; duration_ms ≥ 0.

EXEMPLOS DE JSON

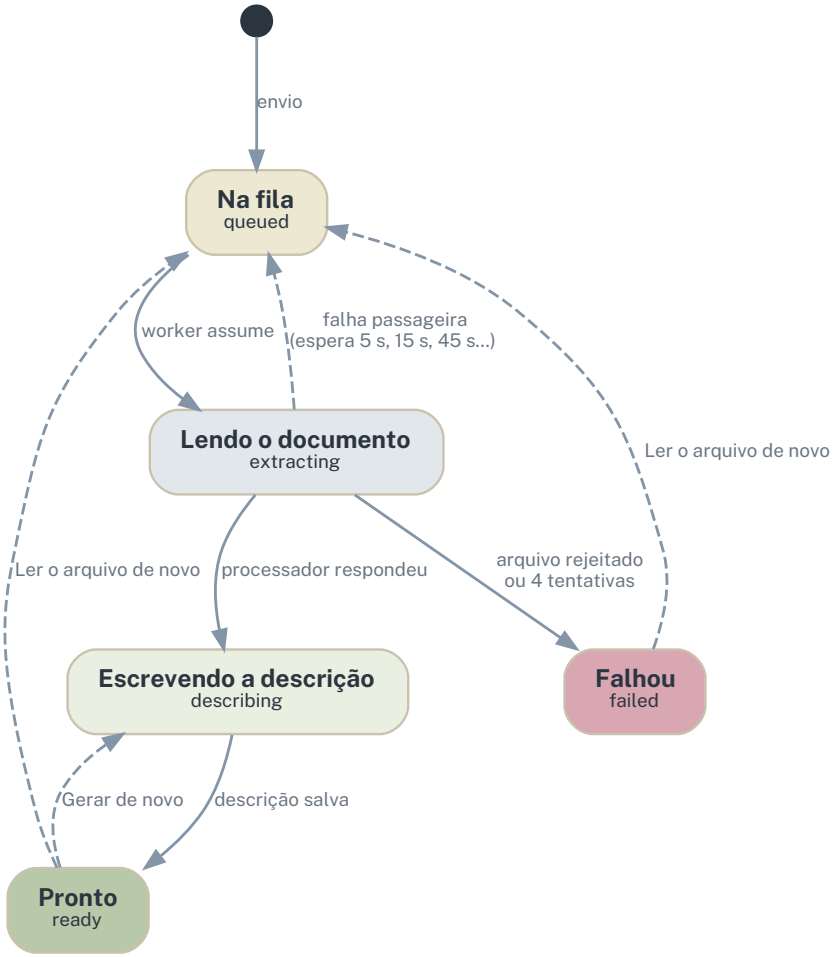
Campo	Exemplo	Quando
details (extract)	`{"pages": 8, "words": 1028, "processor_ms": 442}`	Leitura concluída.
details (retrying)	`{"retry_in_s": 15}`	Espera até a próxima tentativa.
details (describe)	`{"provider": "anthropic", "model": "claude-haiku-5-5", "fallback": false}`	fallback = true quando o provedor externo falhou e o resumo local foi usado.

CAPÍTULO 5

Máquinas de estado

Documento

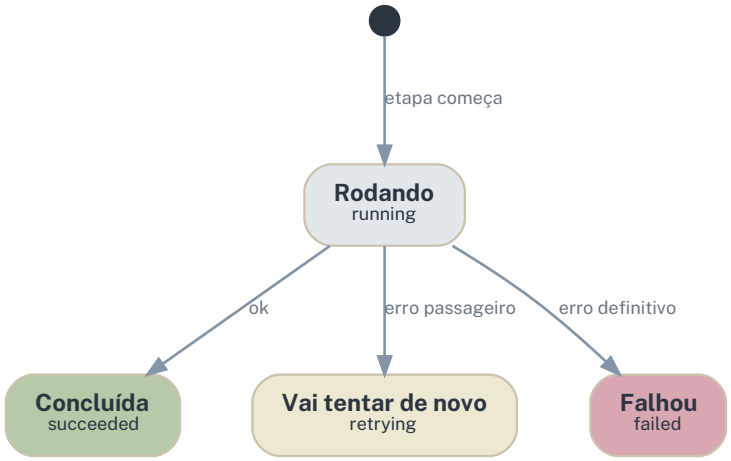
A coluna `library_document.status` segue o pipeline. Setas tracejadas são caminhos de volta: retentativa, ações do dono no detalhe do documento e a varredura de documentos parados.



Transição	Quem faz	O que muda
→ queued	Envio (web)	Linha criada; arquivo na pasta; `XADD docs:extract`.
queued → extracting	Worker	`attempts + 1`, `status_detail = ''`; cria o job.
extracting → describing	Worker	Metadados, capa, sumário, texto e busca gravados; `attempts = 0`; `XADD docs:describe`.
extracting → queued	Worker	Erro passageiro (processador fora do ar, sem vaga, rede): `status_detail = "Nova tentativa em N s"` e entrada no ZSET `docs:delayed`. Espera 5 s, 15 s, 45 s, 135 s (máximo 300 s).
extracting → failed	Worker	Arquivo rejeitado (senha, corrompido, formato) ou 4 tentativas sem sucesso (`PIPELINE_MAX_ATTEMPTS`).

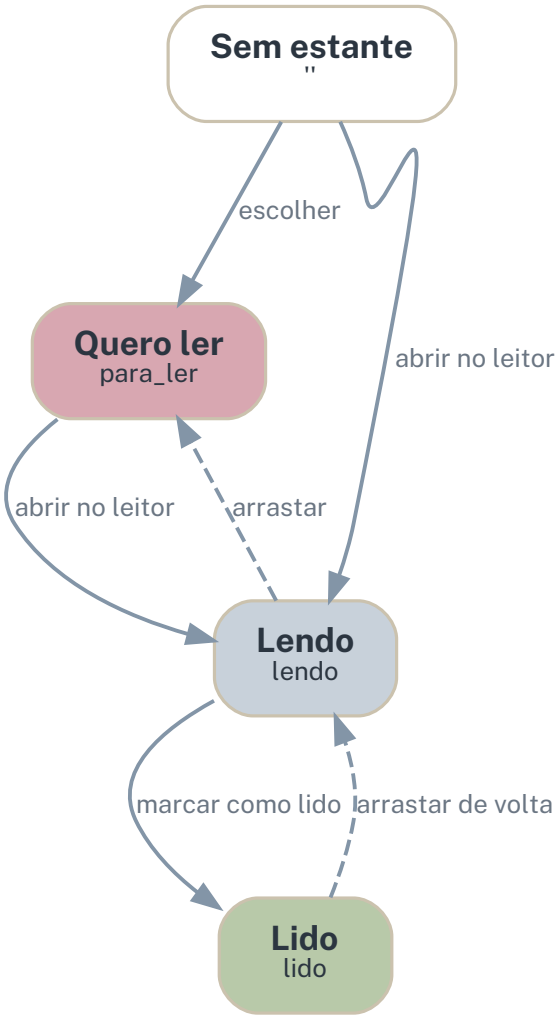
Transição	Quem faz	O que muda
describing → ready	Worker	Descrição, etiquetas da IA, idioma e busca; `processed_at` na primeira vez. Se a IA externa falhar, usa o resumo local: esta etapa não falha.
ready/failed → queued	Dono: "Ler o arquivo de novo"	`attempts = 0`; volta à fila de leitura.
ready → describing	Dono: "Gerar de novo"	Volta só para a fila de descrição.
parado → fila	Varredura do worker	Documento em queued/extracting/describing sem mudança há 30 min (`PIPELINE_STALE_AFTER`) volta para a fila da etapa.

Job



Cada tentativa começa como `running`. Ao terminar, grava `finished_at`, `duration_ms`, o erro (se houver) e os `details`. `retrying` significa que o documento voltou para a fila com espera.

Estante



`library_readingstate.shelf` começa vazio. Abrir o documento no leitor move de vazio ou “Quero ler” para “Lendo” e grava `started_at`. Marcar como lido grava `finished_at`; mover de volta limpa essa data. Na tela Estantes o usuário arrasta o cartão para qualquer coluna.

CAPÍTULO 6

Busca em texto completo

Cada documento tem um `search_vector` com pesos. O worker o recalcula ao terminar cada etapa; a API recalcula quando o dono edita título, autor, descrição ou etiquetas.

Peso	Conteúdo
A	Título.
B	Autor; descrição por IA, descrição do usuário e etiquetas.
C	Texto extraído (primeiros 300 mil caracteres, para respeitar o limite de 1 MB do `tsvector`).

Como o vetor é calculado

```
UPDATE library_document d SET search_vector =
    setweight(to_tsvector('pt_unaccent', coalesce(d.title, '')), 'A') ||
    setweight(to_tsvector('pt_unaccent', coalesce(d.author, '')), 'B') ||
    setweight(to_tsvector('pt_unaccent', coalesce(d.summary, '') || ' ' || coalesce(d.description,
'') || ' ' ||
        coalesce((SELECT string_agg(t.name, ' ') FROM library_documenttag dt
                  JOIN library_tag t ON t.id = dt.tag_id WHERE dt.document_id = d.id), '')), 'B') ||
    setweight(to_tsvector('pt_unaccent', coalesce(
        (SELECT left(c.text, 300000) FROM library_documentcontent c WHERE c.document_id = d.id),
'')), 'C')
WHERE d.id = :id;
```

Como a busca consulta

A caixa de busca aceita a sintaxe de `websearch_to_tsquery`: palavras soltas, `"frase exata"`, `-palavra` para excluir e `or`. Um documento entra no resultado se casar com a busca textual **ou** se o título tiver similaridade de trigramas acima de 0,25 (pega erros como “postgress”). A ordem é relevância (`ts_rank`), depois similaridade, depois data de envio.

```
SELECT d.id, d.title,
       ts_rank(d.search_vector, websearch_to_tsquery('pt_unaccent', 'banco de dados')) AS rank,
       similarity(d.title, 'banco de dados') AS sim
FROM library_document d
WHERE (d.owner_id = :usuario OR d.visibility = 'shared')
  AND (d.search_vector @@ websearch_to_tsquery('pt_unaccent', 'banco de dados')
       OR similarity(d.title, 'banco de dados') > 0.25)
ORDER BY rank DESC, sim DESC, d.created_at DESC
LIMIT 48;
```

Trechos encontrados

Para os documentos da página atual, a API pede o trecho onde a busca aparece. O texto é escapado antes de virar HTML e só as marcações `<mark>` são acrescentadas.

```
SELECT c.document_id,
       ts_headline('pt_unaccent', left(c.text, 100000),
                   websearch_to_tsquery('pt_unaccent', 'banco de dados'),
                   'MaxFragments=1, MaxWords=26, MinWords=12')
FROM library_documentcontent c
WHERE c.document_id = ANY(:ids);
```

POR QUE PT_UNACCENT

A configuração tira os acentos e reduz as palavras ao radical português: *decisões*, *decisao* e *decidir* se aproximam. Sem ela, quem digita sem acento não acha nada.

CAPÍTULO 7

Um documento passo a passo

1. Envio

1. O web grava o arquivo em `/data/tmp/<uuid>.part` calculando o SHA-256 e checa o tamanho (`APP_MAX_UPLOAD_MB`).
2. Detecta o tipo pelo conteúdo (`%PDF-` ou ZIP com `word/document.xml`). Arquivos `.doc` antigos são recusados.
3. Se já existe `(owner_id, sha256)`, devolve o documento existente e apaga o temporário.
4. Numa transação: `INSERT library_document` (status `queued`, título do nome do arquivo), move o arquivo para `documents/<id>/original.<tipo>` e, se veio uma coleção, `INSERT library_collectionitem`.
5. Depois do `COMMIT`: `XADD docs:extract` e `PUBLISH events:user:<dono>` (`document.created`).

2. Leitura do arquivo (processador Java)

1. Um worker lê a mensagem com `XREADGROUP` no grupo `workers` e assume o documento: `status = 'extracting'`, `attempts + 1`.
2. `INSERT processing_job` (stage `extract`, status `running`) e aviso pelo pub/sub.
3. Chama `POST /v1/process` no processador, que escreve `cover.jpg`, `reader.html` e `media/` na pasta.
4. Numa transação: atualiza páginas, palavras, idioma, sumário, metadados, capa (`cover_version + 1`), título e autor (se não estiverem travados), `status = 'describing'`; grava o texto em `library_documentcontent` (insere ou atualiza); recalcula o `search_vector`. No DOCX, higieniza o HTML e grava `reader.safe.html`.
5. Fecha o job como `succeeded`, faz `XADD docs:describe` e `XACK` da mensagem.

3. Descrição por IA

1. Outro (ou o mesmo) worker assume pela fila `docs:describe` e cria o job `describe`.
2. Lê o texto, pede a descrição ao provedor configurado (local, Anthropic ou API compatível com OpenAI) e, se ele falhar, usa o resumo local.
3. Numa transação: grava `summary`, `summary_source`, `summary_model`, `summary_at`, idioma, `status = 'ready'`, `processed_at`; troca as etiquetas de origem `ia` (as do usuário ficam); recalcula a busca.
4. Fecha o job e publica o evento: os navegadores abertos atualizam o cartão e a capa aparece.

Chaves do Redis

Chave	Tipo	Para quê
`docs:extract`	stream	Fila da leitura do arquivo. Campo `document_id`; tamanho aproximado limitado a 10 mil entradas.
`docs:describe`	stream	Fila da descrição por IA.

Chave	Tipo	Para quê
`workers`	grupo	Grupo de consumidores das duas filas. Mensagem sem confirmação há mais de 10 min (`PIPELINE_VISIBILITY_TIMEOUT`) é reassumida com `XAUTOCLAIM` .
`docs:delayed`	sorted set	Retentativas com espera; membro JSON `{"stage", "document_id"}` , score = horário (epoch). Os workers movem os vencidos para o stream.
`events:user:<id>`	pub/sub	Avisos para os navegadores daquela conta (`document.created` , `document.updated` , `document.deleted` , `reading.updated` , `collections.changed`).
`events:shared`	pub/sub	Avisos de documentos compartilhados, para todas as contas.
`workers:<nome>`	hash, TTL 30 s	Batimento de cada worker (etapas, threads, ocupados, processados, falhas). Some sozinho se o worker morrer.
`login:fail:<ip>:<email>`	string, TTL 15 min	Tentativas de login erradas; a partir de 5, o login fica bloqueado até expirar.
`cache:1:*`	strings	Cache do Django (prefixo `cache`), incluindo a cópia das sessões.

A mensagem de evento só avisa o que mudou (`{"type", "id", "status", "owner_id"}`); o navegador busca o documento de novo pela API, com as próprias permissões.

CAPÍTULO 8

Consultas úteis

Para abrir o `psql` no Docker: `docker compose exec postgres psql -U lume -d lume`.

Documentos por estado

```
SELECT status, count(*) FROM library_document GROUP BY status ORDER BY status;
```

Falhas recentes

```
SELECT d.title, j.stage, j.attempt, j.error, j.started_at
FROM processing_job j JOIN library_document d ON d.id = j.document_id
WHERE j.status = 'failed'
ORDER BY j.started_at DESC LIMIT 20;
```

Tempo médio por etapa nas últimas 24 horas

```
SELECT stage, count(*) AS etapas, round(avg(duration_ms)) AS media_ms,
       percentile_disc(0.95) WITHIN GROUP (ORDER BY duration_ms) AS p95_ms
FROM processing_job
WHERE status = 'succeeded' AND started_at > now() - interval '24 hours'
GROUP BY stage;
```

Maiores arquivos

```
SELECT d.title, d.kind, pg_size_pretty(d.size_bytes) AS tamanho, u.email AS dono
FROM library_document d JOIN accounts_user u ON u.id = d.owner_id
ORDER BY d.size_bytes DESC LIMIT 10;
```

Leitura por pessoa

```
SELECT u.name,
       count(*) FILTER (WHERE r.shelf = 'lendo') AS lendo,
       count(*) FILTER (WHERE r.shelf = 'lido')  AS lidos,
       count(*) FILTER (WHERE r.favorite)        AS favoritos,
       max(r.last_opened_at)                     AS ultima_leitura
FROM library_readingstate r JOIN accounts_user u ON u.id = r.user_id
GROUP BY u.name ORDER BY ultima_leitura DESC NULLS LAST;
```

Etiquetas mais usadas

```
SELECT t.name, count(*) AS documentos,
       count(*) FILTER (WHERE dt.source = 'ia') AS sugeridas_pela_ia
FROM library_documenttag dt JOIN library_tag t ON t.id = dt.tag_id
GROUP BY t.name ORDER BY documentos DESC LIMIT 20;
```

Documentos parados há mais de 30 minutos

```
SELECT id, title, status, attempts, status_detail, updated_at
FROM library_document
WHERE status IN ('queued', 'extracting', 'describing')
  AND updated_at < now() - interval '30 minutes';
```

CAPÍTULO 9

Como preparar o banco

Com Docker (recomendado)

Não há nada a fazer. O `docker compose up` sobe o PostgreSQL 16 com o usuário e o banco de `DB_USER`, `DB_NAME` e `DB_PASSWORD` (o `iniciar.cmd` / `iniciar.sh` gera a senha), e o serviço `web` roda `python manage.py migrate` antes de abrir a porta. Como o usuário criado pela imagem é superusuário do banco, a migração cria as extensões sozinha. A porta 5432 só é publicada em `127.0.0.1`.

PostgreSQL próprio

```
-- como superusuário (postgres)
CREATE ROLE lume LOGIN PASSWORD 'uma-senha-forte';
CREATE DATABASE lume OWNER lume ENCODING 'UTF8';
\c lume
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS unaccent;
```

Criar as extensões antes é necessário quando o usuário da aplicação não é superusuário (a migração usa `CREATE EXTENSION IF NOT EXISTS` e passa direto se elas já existem). Depois, aponte a aplicação para o banco:

Variável	Padrão	Uso
`DB_HOST`	`localhost` (`postgres` no Docker)	Servidor.
`DB_PORT`	`5432`	Porta.
`DB_NAME` / `DB_USER`	`lume`	Banco e usuário.
`DB_PASSWORD`	—	Senha (obrigatória no Docker).
`DB_CONN_MAX_AGE`	`60`	Segundos que uma conexão fica aberta para reuso (com verificação de saúde).

```
cd web
python manage.py migrate
python manage.py createuser --email voce@empresa.com --name "Seu Nome" --admin
```

Banco gerenciado

Serviços como Amazon RDS, Google Cloud SQL, Azure Database ou Supabase funcionam: `pg_trgm` e `unaccent` estão na lista de extensões permitidas de todos eles. Crie as extensões com o usuário administrador do serviço antes da primeira migração, use conexão com TLS quando o provedor exigir e lembre que a busca roda inteira dentro do PostgreSQL (não há serviço de busca à parte).

Redis

Redis 7 (`REDIS_URL`, padrão `redis://localhost:6379/0`). No Docker ele roda com `appendonly yes` para não perder filas num reinício. Sem persistência também funciona: os workers recriam os grupos e a varredura devolve à fila o que estava pendente. Os testes usam o banco 15 do Redis (`TEST_REDIS_URL`) e o apagam a cada teste; nunca aponte os testes para o Redis de produção.

CAPÍTULO 10

Backup, retenção e problemas comuns

Backup

Um backup completo tem duas partes, feitas no mesmo momento: o banco e o volume de arquivos.

```
# banco (formato custom, compactado)
docker compose exec -T postgres pg_dump -U lume -d lume -Fc > lume-$(date +%F).dump

# arquivos dos documentos (capas, originais, HTML de leitura)
docker run --rm -v lume_documents:/data -v "$PWD":/backup alpine \
  tar czf /backup/documentos-$(date +%F).tgz -C /data .
```

Restauração

```
docker compose up -d postgres
docker compose exec -T postgres pg_restore -U lume -d lume --clean --if-exists <
lume-2026-10-20.dump
docker run --rm -v lume_documents:/data -v "$PWD":/backup alpine \
  sh -c "cd /data && tar xzf /backup/documentos-2026-10-20.tgz"
docker compose up -d
```

O `pg_dump` leva junto as extensões e a configuração `pt_unaccent`. Se restaurar só o banco, os documentos aparecem sem capa e sem arquivo; nesse caso, use “Ler o arquivo de novo” depois de devolver os originais.

Retenção e tamanho

- O Lume não apaga nada sozinho. O que mais cresce é `library_documentcontent` (texto completo, até 2 milhões de caracteres por documento) e `processing_job` (duas linhas por documento processado, mais as retentativas).
- O histórico de jobs pode ser enxugado sem efeito no funcionamento: `DELETE FROM processing_job WHERE started_at < now() - interval '90 days';`
- Para ver o tamanho das tabelas: `SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) FROM pg_catalog.pg_statio_user_tables ORDER BY pg_total_relation_size(relid) DESC;`
- O volume de arquivos ocupa mais ou menos a soma de `size_bytes` mais 40 a 80 KB de capa por documento.

Problemas comuns

Sintoma	Causa provável e solução
`permission denied to create extension "pg_trgm"`	O usuário da aplicação não pode criar extensões. Crie `pg_trgm` e `unaccent` como superusuário e rode `migrate` de novo.
`text search configuration "pt_unaccent" does not exist`	A migração `library.0001` não foi aplicada nesse banco. Rode `python manage.py migrate`.

Sintoma	Causa provável e solução
Documentos parados em “Na fila”	Worker ou processador fora do ar. Veja `docker compose ps` e `docker compose logs worker processor`. Quando voltarem, a fila anda; documentos esquecidos voltam sozinhos em até 30 min.
“Já estava na sua biblioteca” ao enviar	O mesmo arquivo (mesmo SHA-256) já existe para essa conta. É a restrição `(owner_id, sha256)`.
A busca não acha um documento recém-enviado	O vetor é calculado ao fim da leitura. Espere o estado “Pronto”.
`duplicate key value violates unique constraint "accounts_user_email_unique"`	Já existe conta com esse e-mail (com outras maiúsculas).
O Redis foi apagado	Nada a fazer: os workers recriam os grupos e a varredura recoloca os documentos pendentes na fila.
`DELETE` manual falha com violação de chave estrangeira	As cascatas são feitas pelo Django. Apague pela interface/API ou apague antes as linhas filhas.

CAPÍTULO 11

Evolução do esquema

O esquema é definido pelos modelos do Django (`web/apps/*/models.py`) e versionado em migrações (`web/apps/*/migrations/`). O banco nunca é alterado à mão.

1. Altere o modelo (por exemplo, uma coluna nova em `Document`).
2. Gere a migração: `python manage.py makemigrations library`. Revise o arquivo gerado.
3. Veja o SQL que será executado: `python manage.py sqlmigrate library 0002`.
4. Aplique: `python manage.py migrate` (no Docker, o serviço `web` aplica sozinho ao subir).
5. Confira o estado: `python manage.py showmigrations`.
6. Rode os testes (`python -m pytest`): eles criam um banco de teste do zero com todas as migrações.

REGRAS

- Nunca edite uma migração que já foi aplicada em algum ambiente; crie uma nova.
- Coluna nova em tabela grande: crie como nula ou com valor padrão e preencha em lotes antes de torná-la obrigatória.
- Índice novo em tabela grande: prefira `AddIndexConcurrently` (de `django.contrib.postgres.operations`) numa migração com `atomic = False`.
- Mudou o cálculo da busca? Recalcule os vetores depois do deploy (por exemplo, com “Ler o arquivo de novo” ou um comando de manutenção que chame `refresh_search` para cada documento).

Migrações atuais: `accounts.0001_initial`, `library.0001_initial` (com as extensões e a configuração `pt_unaccent`) e `processing.0001_initial`, além das migrações dos apps do Django (`auth`, `contenttypes`, `sessions`).